

Governança de IA do MLX Pilot

Última atualização: 2026-08-02

1. Objetivo

Este documento define os princípios, responsabilidades, controles e procedimentos utilizados pelo MLX Pilot para garantir o uso seguro, transparente, privado e auditável de modelos de inteligência artificial.

A governança descrita aqui se aplica ao produto real implementado neste repositório, incluindo chat com LLM, agente multi-turn, tool-calling, skills, memória/contexto, providers locais e remotos, escrita em arquivos, execução de comandos, conexões externas, cofre de segredos e logs de auditoria.

Este documento não substitui parecer jurídico. Ele estabelece regras técnicas e operacionais para orientar desenvolvimento, configuração, uso, revisão acadêmica e resposta a incidentes.

2. Escopo da IA

O MLX Pilot possui mais de uma superfície de IA. Cada superfície tem riscos e controles próprios.

Superfície	Onde aparece	Risco principal	Controles aplicáveis
Chat com LLM	/chat, /chat/stream, aba Chat	Resposta incorreta, envio de prompt a provider configurado	Transparência de provider/modelo, seleção local-first, aviso para provider remoto
Agente multi-turn	POST /agent/run	O modelo pode solicitar ferramentas em cadeia	AgentLoop, limite de iterações, PolicyEngine, ApprovalService, AuditLog
Tool-calling	read_file, write_file, edit_file, list_dir, exec, message, sessions_*, memory_*	Leitura/escrita indevida, envio de mensagens, execução de comando	Catálogo de tools, ToolRisk, profiles, sandbox, approvals e auditoria
Skills	SKILL.md no workspace	Ampliação de capacidades do agente	Capabilities declarativas, elegibilidade, enable/disable, SHA-256 e pins

Providers locais	mlx, llamacpp, ollama	Consumo local de recursos, comportamento do modelo	Preferência local-first, limites de contexto, logs locais
Providers remotos	openai, anthropic, groq, openrouter, deepseek, custom	Envio de prompts/contexto a terceiros	Aviso explícito, API key via vault, minimização de contexto
Memória e contexto	MemoryStore, SessionStore, context budget	Retenção de informações sensíveis	Armazenamento local, exportação/deleção de sessões, controles de memória
Arquivos do workspace	Tools de filesystem	Vazamento, sobrescrita ou alteração acidental	Sandbox de path, paths sensíveis, owner-only mode, read-only/dry-run
Execução de comandos	exec	Comandos destrutivos ou escalada local	Denylist, safe bins, timeout, approval obrigatória por risco
Rede e serviços externos	Providers HTTP, Brave, channels, message	Egress não autorizado e envio de dados	Allowlist de domínios, airgapped mode, bloqueio de IP direto
Segredos	API keys, tokens, channel credentials, skill env	Exposição de credenciais	SecretsVault, api_key_ref, env_refs, redação em logs

3. Princípios

1. Privacidade por padrão: o sistema deve processar localmente sempre que possível e enviar a terceiros apenas o necessário para a tarefa aprovada.
2. Local-first: providers locais (mlx, llamacpp, ollama) devem ser preferidos quando atendem ao caso de uso.
3. Transparência: a UI e as respostas operacionais devem indicar provider, modelo, modo de execução e quando dados podem sair da máquina.
4. Supervisão humana: a IA não deve executar ações sensíveis sem aprovação ou política explícita.
5. Menor privilégio: tools, skills, plugins e channels devem receber apenas as permissões necessárias.
6. Rastreabilidade: prompts, chamadas de provider, tool calls, decisões de policy/approval e erros devem ser auditáveis.
7. Segurança por política: regras devem ser aplicadas antes da execução de qualquer tool.

8. Interrupção e reversibilidade: execuções devem ter limites, timeouts e opções de cancelamento quando suportado.
9. Responsabilidade compartilhada: a equipe implementa controles; administradores configuram políticas; usuários revisam e aprovam ações sensíveis.
10. Minimização: prompts, memória, logs e contexto enviado a modelos devem conter somente o necessário.

4. Classificação de risco das ações

O MLX Pilot já possui um catálogo de tools com ToolRisk (low, medium, high, critical). A governança operacional deve seguir a matriz abaixo.

Nível	Exemplos no MLX Pilot	Comportamento esperado
Baixo	read_file, list_dir, sessions_list, sessions_history, sessions_status, memory_search, memory_get, responder perguntas	Pode ser automático quando permitido pelo profile e pela policy
Médio	sessions_spawn, sessions_send, criação de contexto local, anexar mensagem a sessão local	Pode executar automaticamente em ambiente controlado, mas deve ser auditado e respeitar execution_mode
Alto	write_file, edit_file, message, uso de provider remoto com contexto sensível, envio de mensagens por channels	Requer aprovação humana ou política explícita; deve registrar parâmetros resumidos, decisão e resultado
Crítico	exec, acesso a segredos, tentativa de acesso fora do workspace, paths sensíveis, comandos destrutivos, tool desconhecida	Bloqueado por padrão ou permitido somente com política explícita, aprovação e auditoria completa

Regras adicionais:

- Tools desconhecidas devem ser tratadas como critical e negadas.
- Tools high e critical requerem aprovação pelo fluxo do ApprovalService, salvo quando o administrador tiver configurado explicitamente approval_mode: auto para ambiente controlado.
- approval_mode: auto deve ser usado apenas em desenvolvimento local, testes automatizados ou ambientes isolados.
- approval_mode: deny deve ser usado em ambientes demonstrativos, acadêmicos ou de avaliação quando o objetivo for impedir qualquer ação sensível.
- Operações de escrita e execução devem respeitar execution_mode.

5. Modos de execução e profiles de tools

O projeto implementa ExecutionMode para restringir a capacidade das tools:

Modo	Regra de governança
full	Permite leitura, escrita e execução, desde que policy e approval permitam
read_only	Bloqueia tools mutantes como write_file, edit_file, exec, message, sessions_spawn e sessions_send
dry_run	Simula a ação e registra o que seria feito sem alterar estado, quando a tool suporta esse modo
locked	Bloqueia execução de tools

O projeto também possui profiles de tools:

Profile	Uso recomendado
minimal	Perguntas, leitura controlada e inspeção básica
coding	Desenvolvimento local com filesystem e execução sob aprovação
messaging	Fluxos com channels e envio de mensagens sob aprovação
full	Somente para administradores, testes controlados ou ambientes isolados

As políticas efetivas podem ser consultadas por GET /agent/tools, GET /agent/tools/effective-policy e alteradas por POST /agent/tools/profile ou POST /agent/tools/allow-deney.

6. Supervisão humana

O ApprovalService controla a aprovação de ações sensíveis. Ele possui três modos:

Modo	Comportamento	Uso recomendado
auto	Aprova automaticamente ações que passariam pelo fluxo de aprovação	Apenas desenvolvimento local, smoke tests e ambientes descartáveis
ask	Solicita decisão humana antes da execução	Padrão recomendado para uso interativo
deny	Nega automaticamente ações sensíveis	Demonstrações, avaliação acadêmica, ambientes compartilhados ou produção restrita

Decisões suportadas:

- allow_once: permitir somente esta chamada.
- allow_session: permitir durante a sessão atual.
- allow_always: permitir por padrão parametrizado.
- deny: negar a chamada.

A IA deve pedir aprovação quando:

- uma tool possuir risco high ou critical;
- a tool for exec, mesmo dentro do workspace;
- o comando não estiver na lista de exec_safe_bins;
- a ação escrever ou editar arquivo (write_file, edit_file);
- a ação enviar mensagem externa (message);
- a ação envolver segredo, token, API key, senha ou credential;
- a ação usar rede fora de allowlist ou provider remoto com contexto sensível;
- uma skill tiver hash alterado desde a última carga;
- uma skill exigir capability que não foi declarada ou aprovada;
- a ação tentar acessar path sensível ou fora do workspace;
- houver instalação de dependências, download de artefatos ou configuração externa.

O endpoint POST /agent/approve resolve aprovações pendentes. Cada aprovação deve gerar eventos approval_pending e approval_decision no AuditLog.

7. Privacidade e proteção de dados

7.1 Dados processados

O MLX Pilot pode processar:

- prompts e mensagens do usuário;
- respostas do modelo;
- histórico de sessões;
- arquivos lidos no workspace;
- diferenças ou conteúdo escrito por tools;
- parâmetros de tools;
- resultados resumidos de tools;
- metadados de provider, modelo e tempo de execução;
- memória local compactada;
- chaves de API, tokens e credenciais de channels;
- configurações de skills, plugins, tools e providers.

7.2 Armazenamento local

O armazenamento local inclui:

- SessionStore: mensagens de sessão em JSONL e índice local;
- MemoryStore: artefatos compactos de memória em index.json;
- AuditLog: eventos estruturados em JSONL;
- configuração do agente: provider, modelo, profiles, tools, skills e security config;
- SecretsVault: agent_secrets.key e agent_secrets.v1.json no diretório de configuração.

Na implementação atual, o AuditLog do agente é inicializado em um diretório temporário chamado `mlx-pilot-audit`. Em ambientes de produção ou avaliação formal, a equipe deve configurar um diretório persistente com permissão restrita e política clara de retenção.

7.3 Envio a providers remotos

Quando um provider remoto for utilizado (`openai`, `anthropic`, `groq`, `openrouter`, `deepseek` ou `custom` com endpoint remoto), o sistema deverá informar claramente ao usuário que prompts, contexto, mensagens, tool schemas, trechos de arquivos e resultados de tools poderão ser enviados ao provider configurado.

Regras:

- enviar somente o contexto necessário para a tarefa;
- evitar anexar arquivos inteiros quando um trecho basta;
- não enviar segredos, `.env`, tokens ou credenciais;
- exibir provider e modelo ativos antes ou durante a execução;
- registrar provider/modelo no audit;
- permitir preferência por provider local quando houver alternativa adequada.

`custom` deve ser tratado como remoto exceto quando `base_url` apontar para `localhost`, `127.0.0.1` ou `::1`.

7.4 Segredos e credenciais

Segredos devem ser protegidos pelo `SecretsVault` quando `use_secrets_vault` estiver ativo. A configuração deve persistir referências como `api_key_ref` e `env_refs`, e não o valor em texto claro.

Regras:

- logs não devem conter chaves em texto claro;
- parâmetros com nomes como `api_key`, `token`, `secret`, `password`, `authorization`, `bearer` ou `credential` devem ser tratados como sensíveis;
- skills sem `secrets_access` não devem receber parâmetros que pareçam segredos;
- channel credentials devem usar referências ou armazenamento protegido sempre que possível;
- incidentes de vazamento de segredo exigem rotação imediata.

7.5 Histórico, memória e eliminação

O usuário deve conseguir revisar e exportar sessões por `GET /agent/sessions/{id}/export` e apagar sessões por `DELETE /agent/sessions/{id}`.

A memória local deve seguir minimização:

- armazenar resumos compactos, não conversas completas desnecessárias;
- permitir desabilitar memória pelo Control Plane;
- permitir revisão dos artefatos usados pelo agente;

- prever mecanismo claro de limpeza de memória local quando solicitado pelo usuário.

Se a UI ainda não expuser limpeza detalhada de memória, isso deve ser tratado como requisito de governança pendente antes de uso em ambiente sensível.

7.6 LGPD

O MLX Pilot deve observar, no mínimo, os seguintes princípios da LGPD:

Princípio	Aplicação no MLX Pilot
Finalidade	Explicar por que prompts, arquivos, memória, logs e credenciais são tratados
Adequação	Usar dados apenas para execução, auditoria, segurança e melhoria autorizada
Necessidade	Enviar e armazenar somente o mínimo necessário
Transparência	Informar provider/modelo, logs existentes, memória e envio a terceiros
Segurança	Aplicar vault, sandbox, policies, approvals, allowlists e redação de logs
Prevenção	Bloquear paths sensíveis, comandos destrutivos e egress não autorizado
Livre acesso	Permitir exportação/revisão de sessões e auditoria quando aplicável
Eliminação	Oferecer meios de apagar sessões, memória e segredos
Responsabilização	Manter trilha de auditoria e critérios de decisão

8. Segurança de tools e skills

8.1 Tools

Toda chamada de tool deve passar por:

11. resolução no catálogo de tools;
12. verificação do profile ativo;
13. allow/deny global, por agente e por sessão;
14. execution_mode;
15. capabilities da skill ativa, quando houver;
16. bloqueio de paths sensíveis;
17. sandbox de workspace;
18. política de rede;
19. avaliação de risco;
20. aprovação humana quando aplicável;
21. auditoria.

O sandbox de filesystem deve impedir escape por caminho absoluto, .. ou symlink. Paths sensíveis padrão incluem ~/.ssh/*, ~/.aws/*, ~/.gnupg/*, **/.env e **/.env.*.

Execução de comandos deve respeitar:

- exec_safe_bins;
- exec_deny_patterns;
- timeout;
- limite de output;
- diretório de trabalho no workspace;
- aprovação para risco critical.

Rede deve respeitar:

- airgapped quando ativo;
- egress_allow_domains quando configurado;
- bloqueio de egress direto para IP quando block_direct_ip_egress estiver ativo;
- classificação de risco de tools como message, web_fetch e web_search.

8.2 Skills

Toda skill deve declarar capabilities no SKILL.md. As capabilities reconhecidas incluem:

- fs_read;
- fs_write;
- network;
- exec;
- secrets_access.

Regras:

- skills não devem receber mais capabilities do que precisam;
- skills desconhecidas ou alteradas não devem executar automaticamente;
- cada SKILL.md deve ter hash SHA-256 calculado no load;
- skill_sha256_pins deve bloquear divergências de integridade;
- alteração de hash sem pin deve gerar aviso e revisão;
- require_capabilities deve ser ativado em ambientes mais restritos;
- configurações sensíveis de skills devem usar env_refs no vault;
- instalação de dependências ou downloads por skills deve exigir confirmação.

9. Transparência e limitações

O MLX Pilot deve comunicar claramente que:

- respostas de modelos podem conter erros;
- modelos podem alucinar fatos, caminhos, comandos ou interpretações;

- providers diferentes podem retornar resultados diferentes para o mesmo prompt;
- a saída do modelo não é garantia de veracidade;
- a execução de tools depende de policies, approvals, ambiente e permissão local;
- ações importantes precisam de validação humana;
- usuário e administrador devem saber qual provider, modelo, execution_mode, approval_mode e profile de tools estão ativos;
- provider remoto pode receber prompts, contexto e resultados de tools;

O sistema não deve se apresentar como fonte final para decisões jurídicas, médicas, financeiras, acadêmicas ou de segurança sem revisão humana qualificada.

10. Auditoria

O AuditLog registra eventos em JSONL e deve ser usado como trilha oficial para execuções do agente.

Endpoints relevantes:

- GET /agent/audit: lista eventos recentes com filtros;
- GET /agent/audit/{id}: consulta evento específico;
- GET /agent/audit/export: exporta eventos em JSON;
- GET /agent/sessions/{id}/export: exporta mensagens da sessão.

Campos que devem constar nos eventos de auditoria:

Campo	Finalidade
id	Identificador único do evento
timestamp	Data e hora UTC
session_id	Sessão responsável
event_type	Tipo do evento
provider	Provider utilizado, quando aplicável
model	Modelo utilizado, quando aplicável
tool_name	Tool solicitada/executada
skill_name	Skill associada, quando houver
params_hash	Hash dos parâmetros completos
params_summary	Resumo sanitizado dos parâmetros
result_summary	Resumo do resultado
duration_ms	Duração
decision	allow, deny, allow_always, erro etc.
reason	Motivo da decisão ou bloqueio
policy_rule	Regra final aplicada
policy_trace	Caminho de resolução da policy
tool_risk	Risco catalogado da tool
error	Erro bruto, quando houver
error_summary	Resumo do erro

Eventos auditáveis incluem:

- prompt_built;
- provider_call;
- tool_requested;
- tool_allowed;
- tool_denied;
- tool_executed;
- tool_failed;
- approval_pending;
- approval_decision;
- response_final;
- error;
- skill_loaded;
- skill_rejected;
- session_started;
- session_ended.

Regras de auditoria:

- parâmetros completos não devem ser gravados quando contiverem segredos;
- usar params_hash para correlação e params_summary para leitura humana;
- erros devem ser registrados sem expor tokens;
- eventos de policy devem incluir policy_rule e policy_trace;
- exportações de auditoria devem ser tratadas como dados sensíveis;
- logs devem ter retenção e controle de acesso definidos pelo administrador.

11. Responsabilidades

Papel	Responsabilidade
Equipe de desenvolvimento	Implementar controles, corrigir vulnerabilidades, manter testes de policy/approval/audit e reduzir vazamentos de dados
Administrador	Configurar providers permitidos, API keys, vault, tool profiles, allow/deny, egress, modo airgapped e retenção de logs
Usuário	Revisar respostas, aprovar ou negar ações sensíveis e evitar inserir dados desnecessários
Professor/orientador	Validar critérios acadêmicos, aderência básica à LGPD e coerência da governança com o projeto
Sistema	Aplicar policies, bloquear ações não permitidas, registrar eventos e preservar rastreabilidade
Provider externo	Processar dados conforme seus próprios

12. Gestão de incidentes

Um incidente de IA inclui, por exemplo:

- execução indevida de tool;
- comando destrutivo ou não autorizado;
- vazamento de segredo;
- envio indevido de prompt, arquivo ou contexto a provider remoto;
- envio de mensagem externa para alvo errado;
- bypass de sandbox;
- skill alterada ou maliciosa;
- log contendo dado sensível não redigido;
- resposta que induz o usuário a ação perigosa.

Procedimento:

22. Interromper ou bloquear a execução.
23. Preservar AuditLog, sessão e configuração efetiva.
24. Identificar provider, modelo, skill, tool, parâmetros resumidos e policy aplicada.
25. Avaliar impacto sobre arquivos, segredos, terceiros e dados pessoais.
26. Rotacionar chaves ou tokens expostos.
27. Desabilitar skill/tool/provider envolvido, se necessário.
28. Corrigir a vulnerabilidade ou a regra de policy.
29. Adicionar teste automatizado para evitar regressão.
30. Documentar incidente, causa raiz e mitigação.
31. Comunicar usuários afetados quando houver dado pessoal ou terceiro envolvido.

13. Métricas de governança

Indicadores recomendados:

- número de ações bloqueadas por policy;
- número de aprovações solicitadas;
- percentual de aprovações aceitas, recusadas e expiradas;
- quantidade de chamadas high e critical;
- falhas por provider;
- tempo médio de resposta por provider/modelo;
- erros de tools;
- tentativas de acesso a paths sensíveis;
- comandos negados por exec_deny_patterns;
- tentativas de egress para domínios não permitidos;
- incidentes de segurança;

- quantidade aproximada de dados enviada a providers remotos;
- quantidade de sessões exportadas/deletadas;
- skills bloqueadas por hash divergente;
- uso de approval_mode: auto fora de ambiente controlado.

14. Regras recomendadas por ambiente

Ambiente	Provider	approval_mode	execution_mode	Tool profile	Observações
Desenvolvimento local	Local ou remoto controlado	ask ou auto temporário	full ou dry_run	coding	auto apenas para tarefas repetitivas e sem dados sensíveis
Demonstração acadêmica	Preferencialmente local	ask	read_only ou dry_run	minimal ou coding	Mostrar claramente provider/modelo e audit
Ambiente compartilhado	Local	deny ou ask	read_only	minimal	Bloquear exec, rede externa e escrita por padrão
Produção sensível	Local ou remoto aprovado	ask	read_only por padrão	minimal	Habilitar owner-only, egress allowlist, vault e retenção formal
Airgapped	Local	deny ou ask	read_only/locked	minimal	Bloquear providers remotos e network tools

15. Critérios de aceite para esta governança

- [x] Descrever escopo da IA no MLX Pilot.
- [x] Definir níveis de risco.
- [x] Definir ações que exigem aprovação.
- [x] Documentar tratamento de dados locais e remotos.
- [x] Documentar controle de tools e skills.
- [x] Definir auditoria e resposta a incidentes.
- [x] Definir papéis e responsabilidades.
- [x] Revisar aderência básica à LGPD.

16. Referências Bibliográficas e Normativas

Este documento baseia-se nas seguintes referências e padrões internacionais de governança e segurança de IA:

32. NIST AI Risk Management Framework (AI RMF 1.0): Estrutura desenvolvida pelo National Institute of Standards and Technology para gerenciar riscos individuais e organizacionais associados à IA. Acessar [NIST AI RMF](#)
33. EU AI Act (Regulamento de IA da União Europeia): O primeiro marco regulatório abrangente do mundo para inteligência artificial, estabelecendo obrigações baseadas em níveis de risco. Acessar [EU AI Act](#)
34. ISO/IEC 42001:2023: Norma internacional que especifica requisitos para estabelecer, implementar, manter e melhorar continuamente um Sistema de Gestão de IA. Acessar [ISO 42001](#)
35. OWASP Top 10 for LLM Applications: Guia de referência para desenvolvedores e profissionais de segurança sobre as vulnerabilidades mais críticas em aplicações que utilizam Large Language Models. Acessar [OWASP LLM](#)
36. Lei Geral de Proteção de Dados (LGPD) - Lei nº 13.709/2018: Legislação brasileira que regula as atividades de tratamento de dados pessoais, fundamental para a privacidade em sistemas de IA. Acessar [LGPD](#)
37. OECD AI Principles: Princípios para o desenvolvimento e uso responsável de IA, adotados por países membros da OCDE e parceiros. Acessar [OECD AI](#)
38. Guia de Uso de IA Generativa - Governo Digital (Brasil): Conjunto de diretrizes e recomendações para o uso seguro e ético de ferramentas de IA generativa no âmbito da administração pública federal. Acessar [Guia Gov.br](#)